

Chapter

10

리스트와 키

10.1 리스트와 키란 무엇인가?

10.2 여러 개의 컴포넌트 렌더링하기

10.3 기본적인 리스트 컴포넌트

10.4 리스트의 키에 대해 알아보기

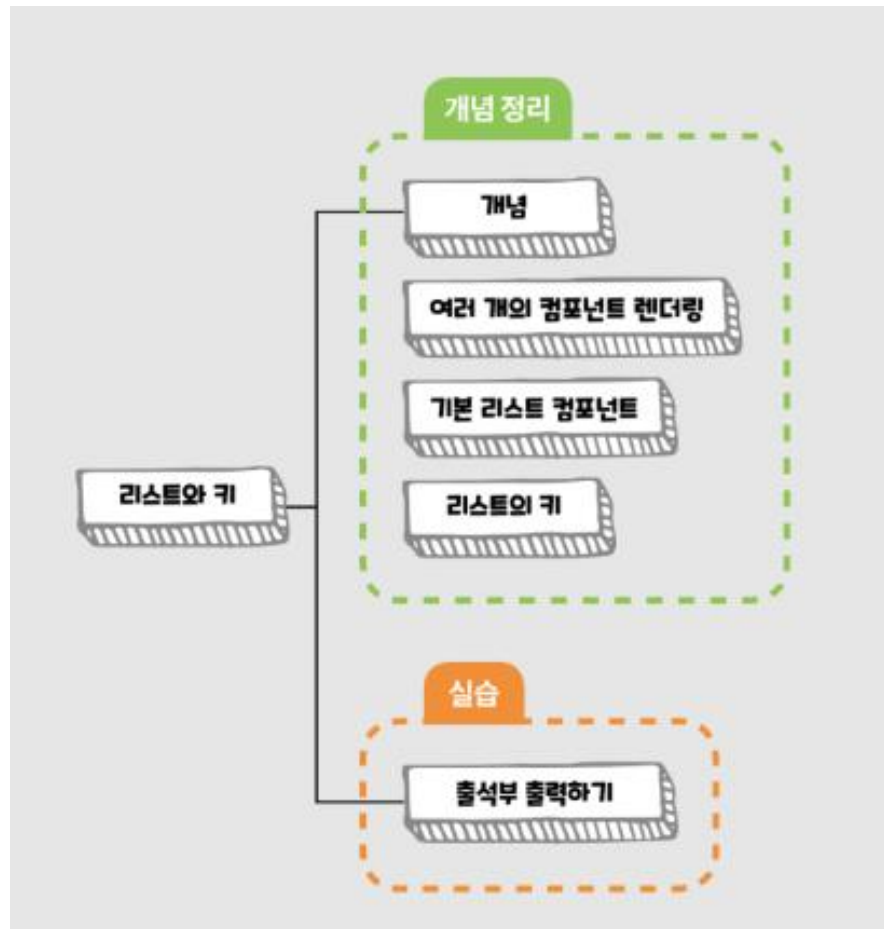
10.5 **실습** 출석부 출력하기

10.6 마치며



Preview

이 장에서는 리스트^{List}와 키^{Key}에 대해서 배워 보도록 하겠습니다.



10.1

리스트와 키란 무엇인가?

리스트는 우리말로 **목록**이라는 뜻을 갖고 있습니다. 컴퓨터 프로그래밍에서는 같은 아이템을 순서대로 모아 놓은 것이 리스트에 해당된다고 볼 수 있습니다. 리스트를 위해 사용하는 자료 구조가 바로 **배열** Array입니다. 배열은 **자바스크립트의 변수나 객체를 하나의 변수로 묶어놓은 것**입니다. 아래는 자바스크립트의 배열을 보여 주고 있습니다.

```
const numbers = [1, 2, 3, 4, 5];
```

그렇다면 키는 무엇일까요? key라는 영단어는 **열쇠**라는 의미를 갖고 있죠. 우리가 알고 있는 열쇠의 특징은 뭐가 있을까요? 바로 열쇠는 모두 다 모양이 다르다는 것입니다. 다시 말해 **열쇠는 모두 각자 고유하다**는 것입니다. 우리 집 열쇠와 옆집 열쇠 모양이 같으면 하나의 열쇠로 두 곳의 문을 모두 열 수 있기 때문에 열쇠의 가치나 의미가 없겠죠. 이와 같은 맥락으로 컴퓨터 프로그래밍에서의 키는 **각 객체나 아이템을 구분할 수 있는 고유한 값**을 의미합니다.

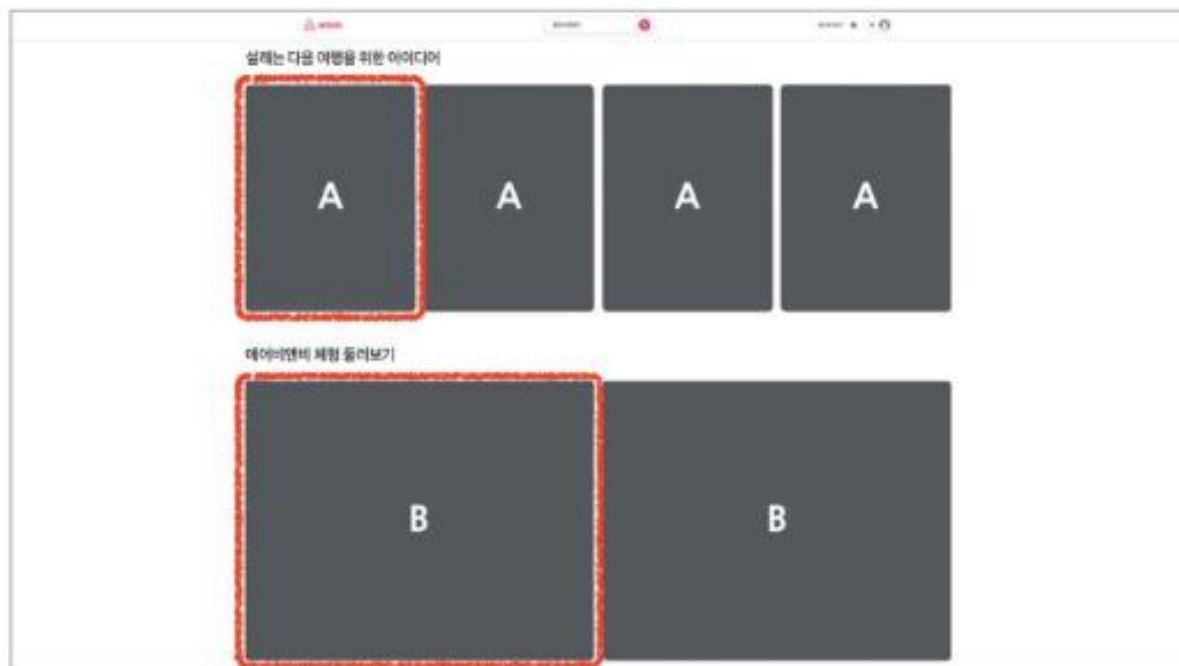
리액트에서는 위에서 나온 배열과 키를 사용하여 반복되는 다수의 엘리먼트를 쉽게 렌더링할 수 있습니다. 그 방법에 대해서 지금부터 하나씩 배워봅시다.



10.2

여러 개의 컴포넌트 렌더링하기

먼저 앞에서 살펴봤던 에어비앤비 첫 화면을 다시 한번 보도록 하겠습니다. 아래 그림을 보면 A라는 컴포넌트와 B라는 컴포넌트가 반복적으로 나오는 것을 볼 수 있습니다. 이렇게 같은 컴포넌트를 화면에 반복적으로 나타내야 할 경우에 이를 코드상에 하나씩 직접 넣는 것은 같은 코드가 반복되기 때문에 굉장히 비효율적입니다. 또한 동적으로 화면의 내용이 바뀌는 경우에 코드를 직접 하나씩 넣는 방식으로는 구현하기가 까다롭습니다.



▶ 컴포넌트 기반 구조



이러한 경우에 사용하는 것이 바로 자바스크립트 배열의 `map()` 함수입니다. `map()` 함수는 영단어 매핑(mapping)을 떠올리면 생각하기 쉽습니다. 매핑이라고 하면 한쪽에 있는 아이템과 다른 한쪽에 있는 아이템을 짝지어 준다는 의미를 가지고 있는데, 여기에서의 매핑도 마찬가지로 배열에 들어 있는 각 변수에 어떤 처리를 한 뒤 리턴하는 것으로 이해하면 됩니다. `map()` 함수를 사용하는 예제 코드를 봅시다.





```
01  const doubled = numbers.map((number) => number * 2);
```

위 코드는 `map()` 함수를 사용하여 `numbers` 배열에 들어 있는 각 숫자에 2를 곱한 값이 들어간 `doubled`라는 배열을 생성하는 코드입니다. 이처럼 `map()` 함수는 배열의 첫 번째 아이템부터 순서대로 각 아이템에 어떠한 연산을 수행한 뒤에 최종 결과를 배열로 만들어서 리턴해 준다고 보면 됩니다. 그렇다면 실제로 리액트에서는 이 `map()` 함수를 어떻게 사용하여 엘리먼트를 렌더링하는지 예제 코드를 보도록 합시다.





```
01  const numbers = [1, 2, 3, 4, 5];  
02  const listItems = numbers.map((number) =>  
03    <li>{number}</li>  
04  );
```

위의 코드를 보면 숫자 1부터 5까지 들어 있는 `numbers`라는 배열이 있고 `map()` 함수를 사용해서 이 배열에 들어 있는 각 숫자를 `<li>` 태그로 감싸 리턴하고 있습니다. JSX에서는 중괄호를 사용하여 자바스크립트 코드를 넣을 수 있기 때문에, 이렇게 하면 각 숫자의 값이 태그 안에 들어가게 됩니다. 이렇게 리턴된 `listItems` 배열은 총 5개의 엘리먼트를 갖고 있게 됩니다. 그리고 이것을 화면에 렌더링하기 위해서 아래와 같이 코드를 작성해 주면 됩니다.





```
01  const root = ReactDOM.createRoot(document.getElementById('root'));
02  root.render(<ul>{listItems}</ul>);
```

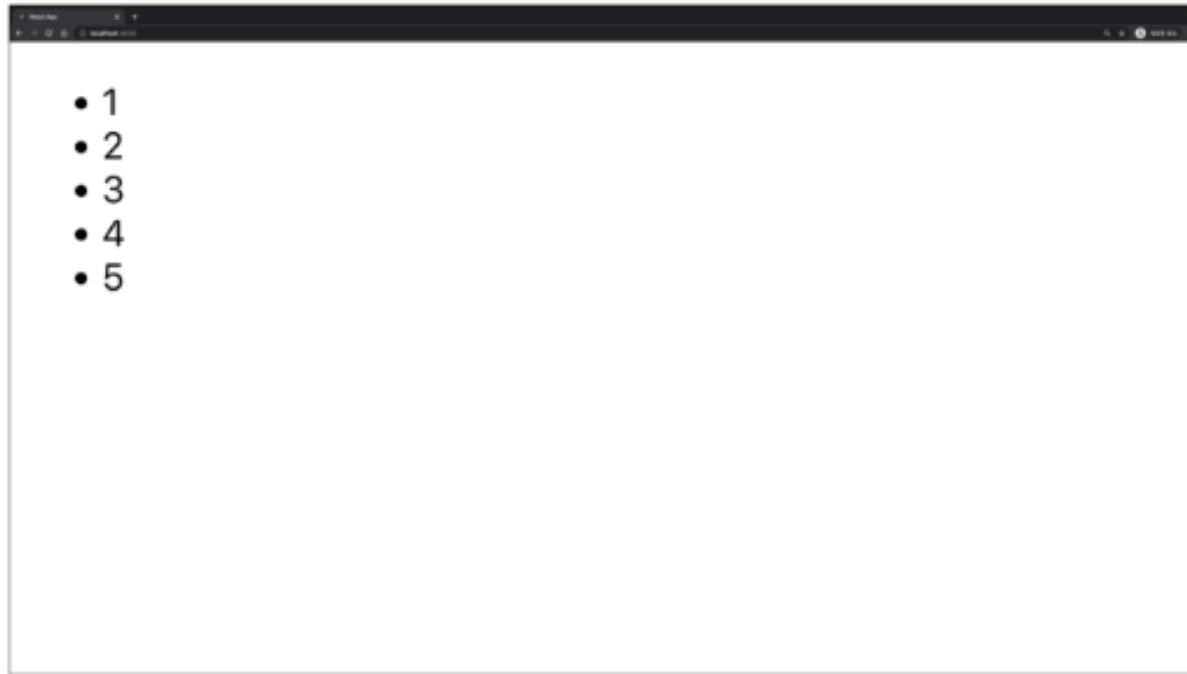
결과적으로 `<li>` 태그가 들어 있는 `listItems` 배열을 `<ul>` 태그로 감싸서 렌더링하게 됩니다. 최종적으로 렌더링되는 코드는 다음과 같습니다.



```
01  const root = ReactDOM.createRoot(document.getElementById('root'));
02  root.render(
03    <ul>
04      <li>{1}</li>
05      <li>{2}</li>
06      <li>{3}</li>
07      <li>{4}</li>
08      <li>{5}</li>
09    </ul>
10  );
```



최종 렌더링 결과는 아래 그림과 같이 1에서 5까지의 숫자가 글머리 기호와 함께 있는 목록으로 출력됩니다.



▶ Number List



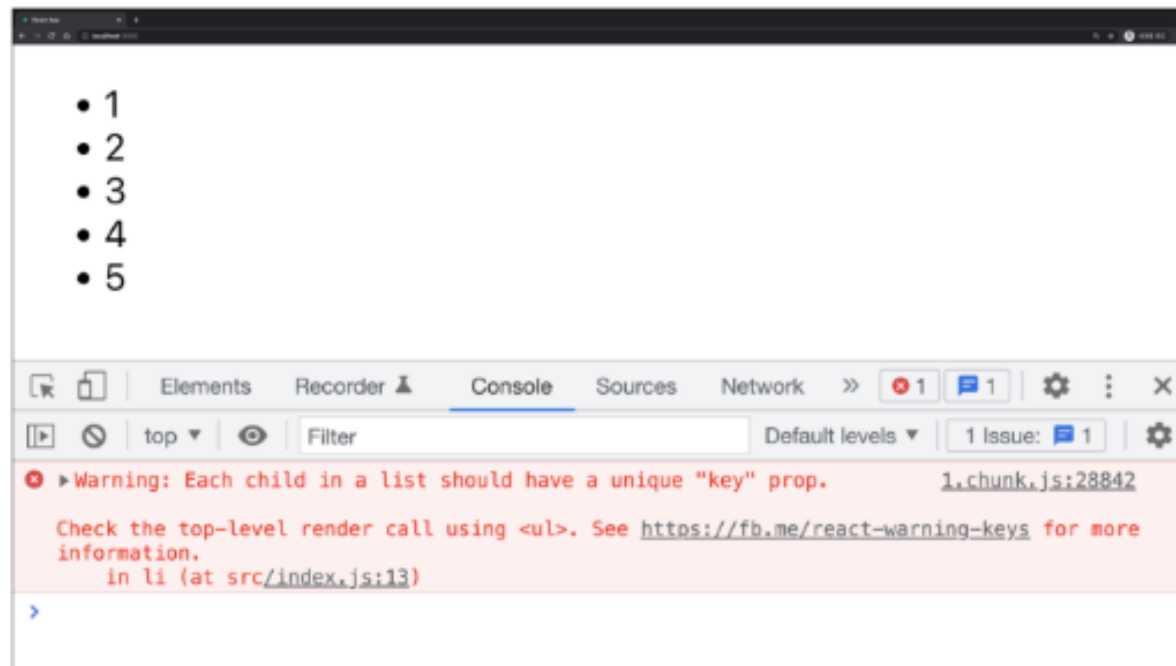
10.3 기본적인 리스트 컴포넌트

이제 위에서 작성한 코드를 별도의 컴포넌트로 분리해 보도록 하겠습니다. 아래 코드는 숫자 목록을 출력하는 `NumberList` 컴포넌트입니다.

```
01 function NumberList(props) {  
02     const { numbers } = props;  
03  
04     const listItems = numbers.map((number) =>  
05         <li>{number}</li>  
06     );  
07  
08     return (  
09         <ul>{listItems}</ul>  
10     );  
11 }  
12  
13 const numbers = [1, 2, 3, 4, 5];  
14 const root = ReactDOM.createRoot(document.getElementById('root'));  
15 root.render(<NumberList numbers={numbers} />);
```



NumberList 컴포넌트는 props로 숫자가 들어 있는 배열인 numbers를 받아서 이를 목록으로 출력합니다. 이 NumberList 컴포넌트를 사용하면 numbers 배열에 숫자가 수십 개 또는 수백 개가 되어도 별도의 코드를 작성할 필요 없이 화면에 렌더링할 수가 있습니다. 그런데 이 코드를 실행해 보면 다음 그림과 같이 개발자 도구의 콘솔 탭에 **리스트 아이템에는 무조건 키가 있어야 한다**는 경고 문구가 나옵니다.



▶ 키 경고 문구

이 경고문이 출력되는 이유는 현재 각 아이템에 키가 없기 때문입니다. 다음절에서 키에 대해서 자세히 배워 보겠습니다.



10.4 리스트의 키에 대해 알아보기

앞에서 키가 갖고 있는 특징은 고유하다는 것이라고 했습니다. 마찬가지로 리액트에서의 키는 **리스트에서 아이템을 구분하기 위한 고유한 문자열**이라고 이해하면 됩니다. 키는 리스트에서 어떤 아이템이 변경, 추가 또는 제거되었는지 구분하기 위해 사용합니다.

우리가 일상생활에서 사용하는 키로는 주민등록번호, 학번, 핸드폰 번호, 여권번호 등이 있습니다. 모두 다 고유한 값이라는 특징을 가지고 있죠. 하지만 고유하다는 특징은 그 범위가 한정되어 있습니다. **리액트에서의 키의 값은 같은 리스트에 있는 엘리먼트 사이에서만 고유한 값이면 됩니다.** 다음 그림을 한번 보도록 합시다.



두 대학교 사이에서는 학번이 같아도 상관없음!

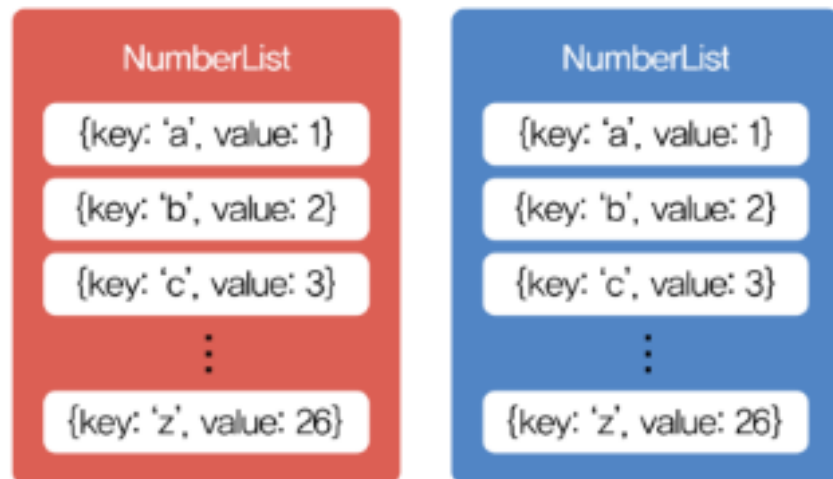


▶ 대학교 학번의 예

위 그림은 A대학교와 B대학교 학생들의 학번을 나타낸 것입니다. 자세히 보면 각 대학교 학생들의 학번이 겹치는 것을 볼 수 있습니다. 하지만 학교별로 구분해서 A대학교 학생들만 보면 학번이 모두 다르고, B대학교 학생들만 볼 때도 각기 학번이 모두 다릅니다. 이처럼 학번은 대학교 내에서 학생을 구분하기 위한 일종의 키이기 때문에 속한 집합 내에서만 고유한 값이 됩니다. 그렇다면 실제 리액트에서는 어떻게 적용될까요?



두 리스트 사이에서는 키가 같아도 상관없음!



▶ `NumberList`의 예

위의 그림은 `NumberList`를 나타낸 것입니다. 각 리스트의 아이템에는 키가 들어 있습니다. 위에 나왔던 대학교 학번 예와 마찬가지로 두 개의 `NumberList` 간에 아이템들의 키값이 중복되는 것을 볼 수 있습니다. 하지만 이 역시 각 리스트 내에서 아이템을 구분하기 위한 용도이기 때문에 속한 리스트 내에서만 고유한 값이면 됩니다.



이제 실제로 고유한 키값을 어떻게 만들어서 사용해야 하는지 알아보겠습니다. 아래 코드는 **키값으로 숫자의 값을 사용**한 것입니다. 이렇게 하면 지금처럼 `numbers` 배열의 숫자들이 중복되지 않는 경우에 정상적으로 작동하지만, 만약 `numbers` 배열에 중복된 숫자가 들어 있다면 키값도 중복되기 때문에 고유해야 한다는 키값의 조건이 충족되지 않습니다. 이러한 경우 실행해 보면 키값이 중복된다는 경고 메시지가 출력됩니다.



```
01  const numbers = [1, 2, 3, 4, 5];
02  const listItems = numbers.map((number) =>
03    <li key={number.toString()}>
04      {number}
05    </li>
06  );
```

다음은 **키값으로 id를 사용하는 방식**이 있습니다. `id`의 의미 자체가 고유한 값이라는 것이기 때문에 키값으로 사용하기에 적합합니다. 이렇게 `id`가 있는 경우에는 보통 이 `id` 값을 키값으로 사용하게 됩니다.





```
01  const todoItems = todos.map((todo) =>
02    <li key={todo.id}>
03      {todo.text}
04    </li>
05  );
```

또한 **키값으로 인덱스(index)를 사용하는 방법**이 있습니다. 이 방법은 `map()` 함수에서 두 번째 파라미터로 제공해 주는 인덱스 값을 키값으로 사용하는 것입니다. 이 인덱스는 배열 내에서 현재 아이템의 인덱스를 의미합니다. 인덱스 값도 고유한 값이기 때문에 키값으로 사용해도 됩니다. 하지만 배열에서 아이템의 순서가 바뀔 수 있는 경우에는 키값으로 인덱스를 사용하는 것을 권장하지 않습니다. 성능에 부정적인 영향을 끼칠 수 있고 컴포넌트의 **state**와 관련하여 문제를 일으킬 수도 있기 때문입니다. 따라서 인덱스를 키값으로 사용하는 것은 아이템들의 고유한 id가 없을 경우에만 사용하는 것이 좋습니다. 참고로 **리액트에서는 키를 명시적으로 넣어 주지 않으면 기본적으로 이 인덱스 값을 키값으로** 사용합니다.



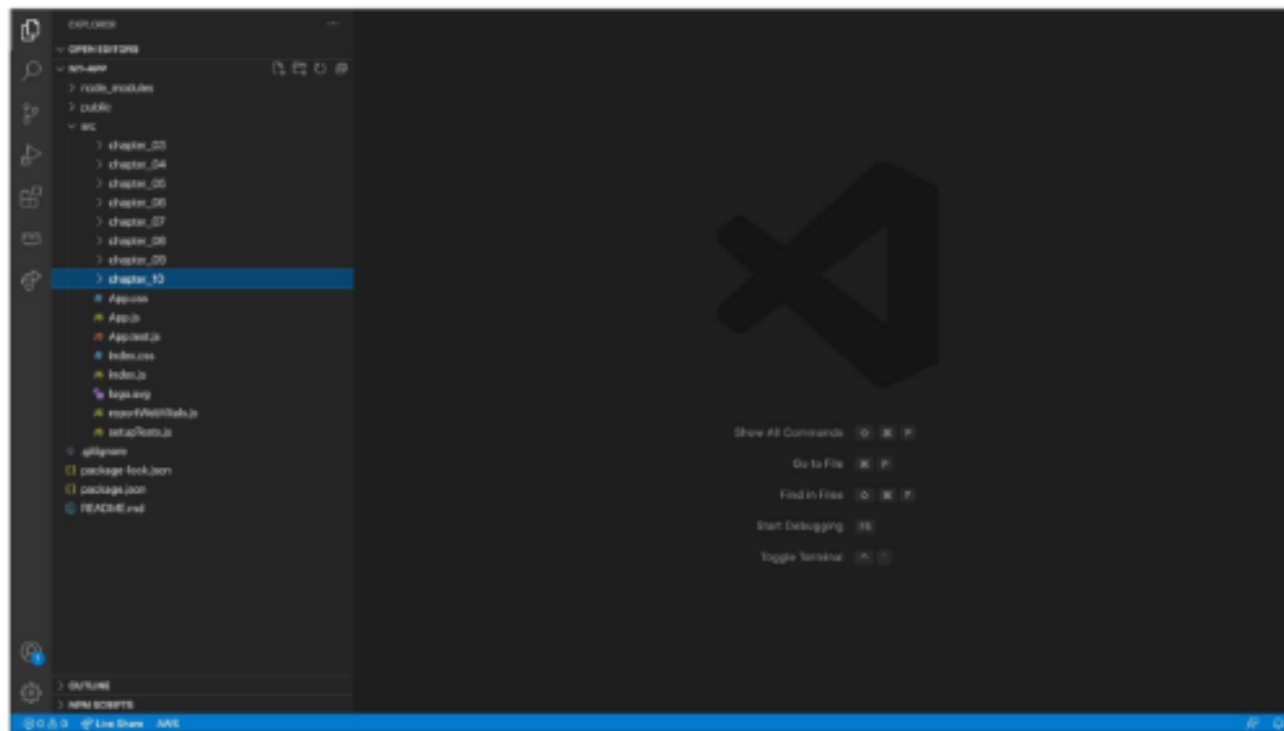


```
01  const todoItems = todos.map((todo, index) =>
02    // 아이템들의 고유한 ID가 없을 경우에만 사용해야 함
03    <li key={index}>
04      {todo.text}
05    </li>
06  );
```

지금까지 다양한 키값의 사용법에 대해서 알아보았습니다. 여기서 꼭 기억해야 할 점은 `map()` 함수 안에 있는 엘리먼트는 꼭 키가 필요하다는 것입니다. 리액트 개발을 처음 하는 분이 가장 많이 하는 실수가 바로 `map()` 함수를 사용하면서 엘리먼트에 키값을 넣지 않는 것입니다. 그렇기 때문에 이 부분을 꼭 잘 기억하고 넘어가기 바랍니다.



이 장 실습에서는 리스트와 키를 사용해서 출석부 컴포넌트를 직접 만들어 보도록 하겠습니다. 먼저 VS Code로 앞 장에서 create-react-app을 이용해 만든 프로젝트를 엽니다. 그리고 아래와 같이 chapter_10이라는 이름으로 폴더를 하나 생성합니다.



▶ 실습 화면 01

그다음 만든 폴더에 AttendanceBook.jsx라는 이름의 파일을 새로 만들고, 아래 코드처럼 AttendanceBook이라는 이름의 함수 컴포넌트를 만듭니다.



```
01 import React from "react";
02
03 const students = [
04   {
05     name: "Inje",
06   },
07   {
08     name: "Steve",
09   },
10   {
11     name: "Bill",
12   },
13   {
14     name: "Jeff",
15   },
16 ];
17
18 function AttendanceBook(props) {
19   return (
20     <ul>
21       {students.map((student) => {
22         return <li>{student.name}</li>;
23       })}
24     </ul>
25   );
26 }
27
28 export default AttendanceBook;
```



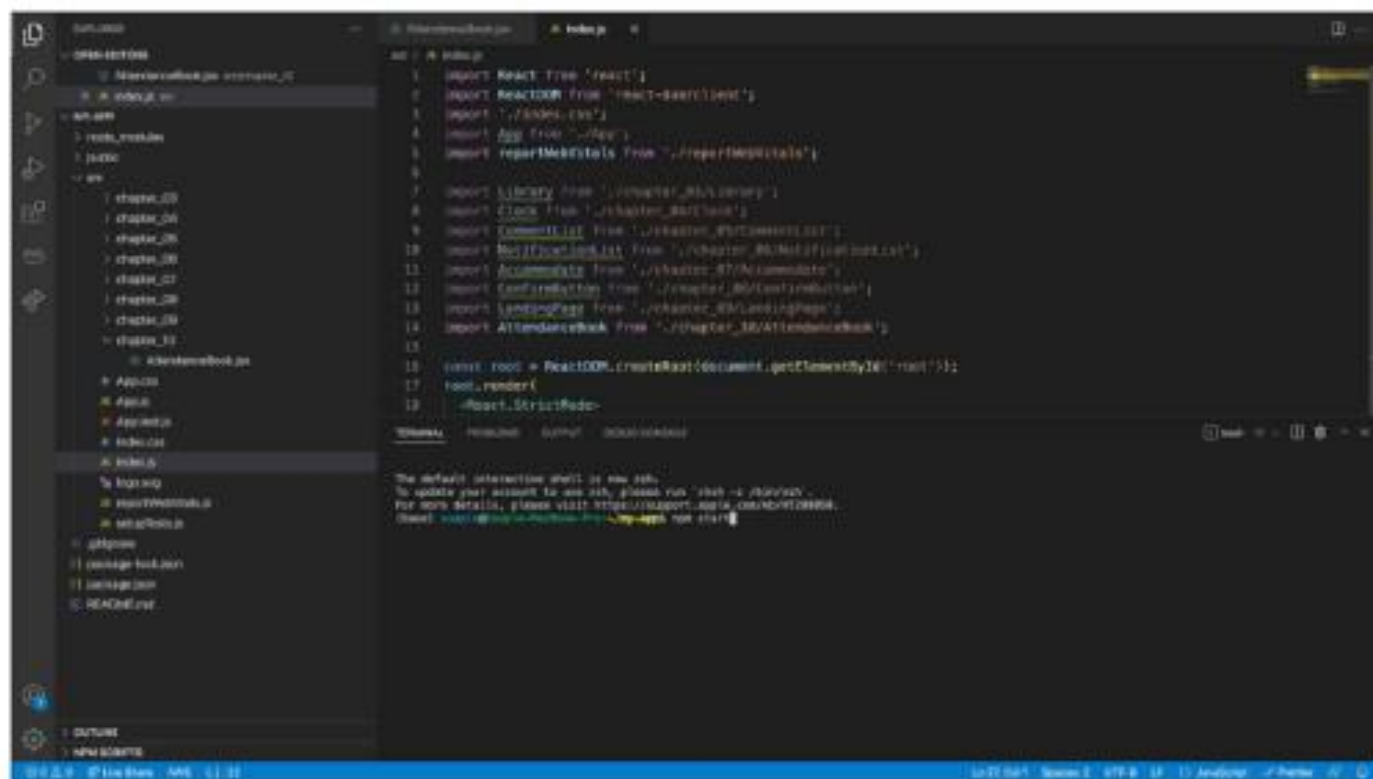
AttendanceBook 컴포넌트는 `students`라는 배열로부터 학생 정보가 담긴 객체를 받아 학생들의 이름을 목록 형태로 출력하는 컴포넌트입니다. 여기에서 배열을 렌더링하기 위해 `map()` 함수를 사용한 것을 볼 수 있습니다.

이제 만든 AttendanceBook 컴포넌트를 실제로 화면에 렌더링하기 위해서 `index.js` 파일을 수정해야 합니다. 다음 코드와 그림에 표시된 부분을 참고하여 새로 만든 AttendanceBook 컴포넌트를 임포트해서 `ReactDOM.createRoot()` 함수로 만든 `root`의 `render()` 함수에 넣어 주는 코드로 변경해 보세요.

```
14 import AttendanceBook from './chapter_10/AttendanceBook';
15
16 const root = ReactDOM.createRoot(document.getElementById('root'));
17 root.render(
18   <React.StrictMode>
19     <AttendanceBook />
20   </React.StrictMode>
21 );
22
```



코드 작성이 끝났으면 실제 리액트 애플리케이션을 실행해 보도록 하겠습니다. VS Code의 상단 메뉴에서 **Terminal > New Terminal**을 눌러 새로운 터미널을 하나 실행시킵니다. 이후 다음 그림처럼 `npm start` 명령어를 실행합니다.



```
1 import React from 'react'
2 import ReactDOM from 'react-dom'
3 import './index.css'
4 import App from './App'
5 import reportWebVitals from './reportWebVitals'
6
7 import {library} from './chapter_01/library'
8 import Clock from './chapter_02/clock'
9 import CommentList from './chapter_03/commentList'
10 import BioFeed from './chapter_04/bioFeed'
11 import Accumulate from './chapter_07/accumulate'
12 import Confirmed from './chapter_08/confirmed'
13 import LandingPage from './chapter_09/landingPage'
14 import AttendanceBook from './chapter_10/attendanceBook'
15
16 const root = ReactDOM.createRoot(document.getElementById('root'))
17 root.render(
18   <React.StrictMode>
```

The default interactive shell is now zsh.
To update your account to use zsh, please run 'chsh -s /bin/zsh'.
For more details, please visit <https://support.apple.com/HT208050>.
Drew: <https://github.com/facebook/create-react-app> npm start

▶ 실습 화면 04



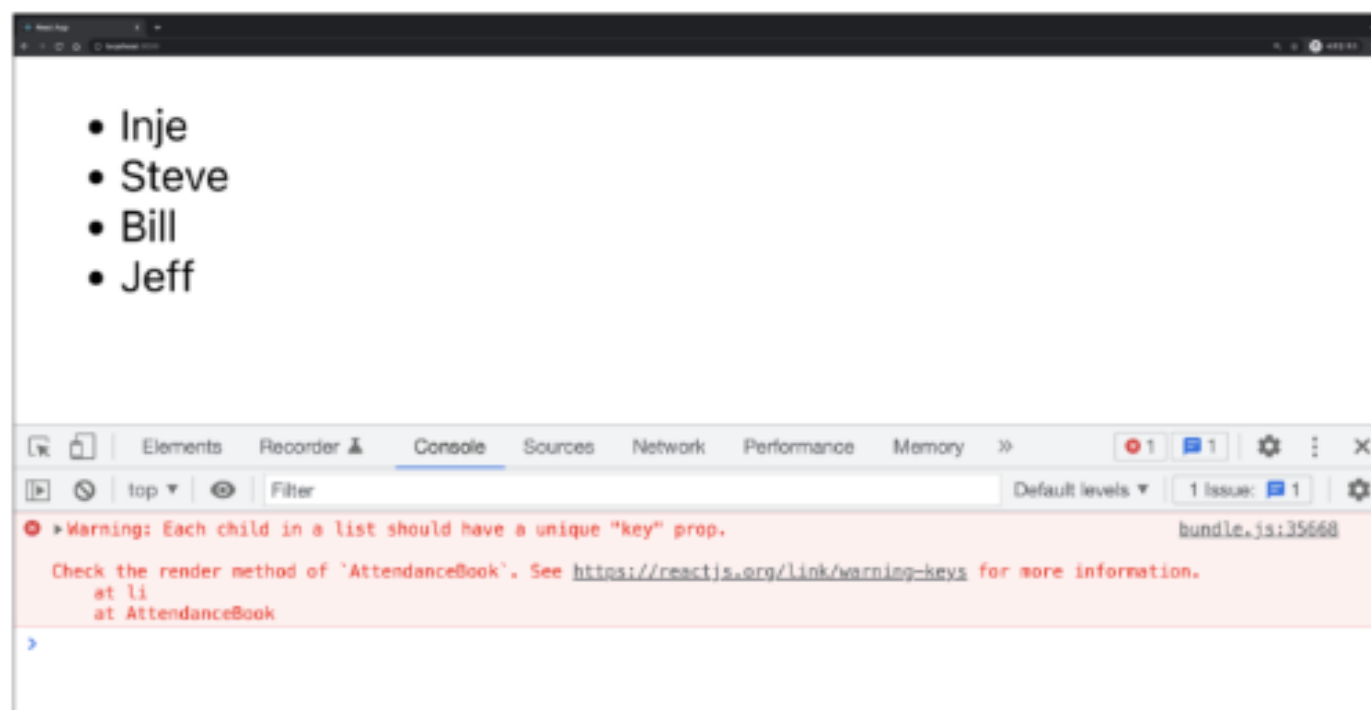
잠시 뒤에 웹브라우저의 새 창이 열리면서 `http://localhost:3000`에 접속되는 것을 볼 수 있습니다. 화면에는 다음 그림과 같이 학생들의 이름이 목록 형태로 출력됩니다.



▶ 실습 화면 05



그런데 여기에서 크롬 개발자 도구를 열어 콘솔 탭에 가보면 빨간 글씨로 경고 문구가 나오는 것을 볼 수 있습니다. `map()` 함수를 사용해서 목록을 렌더링했는데 각 엘리먼트에 키가 빠져 있기 때문입니다.



▶ 실습 화면 06



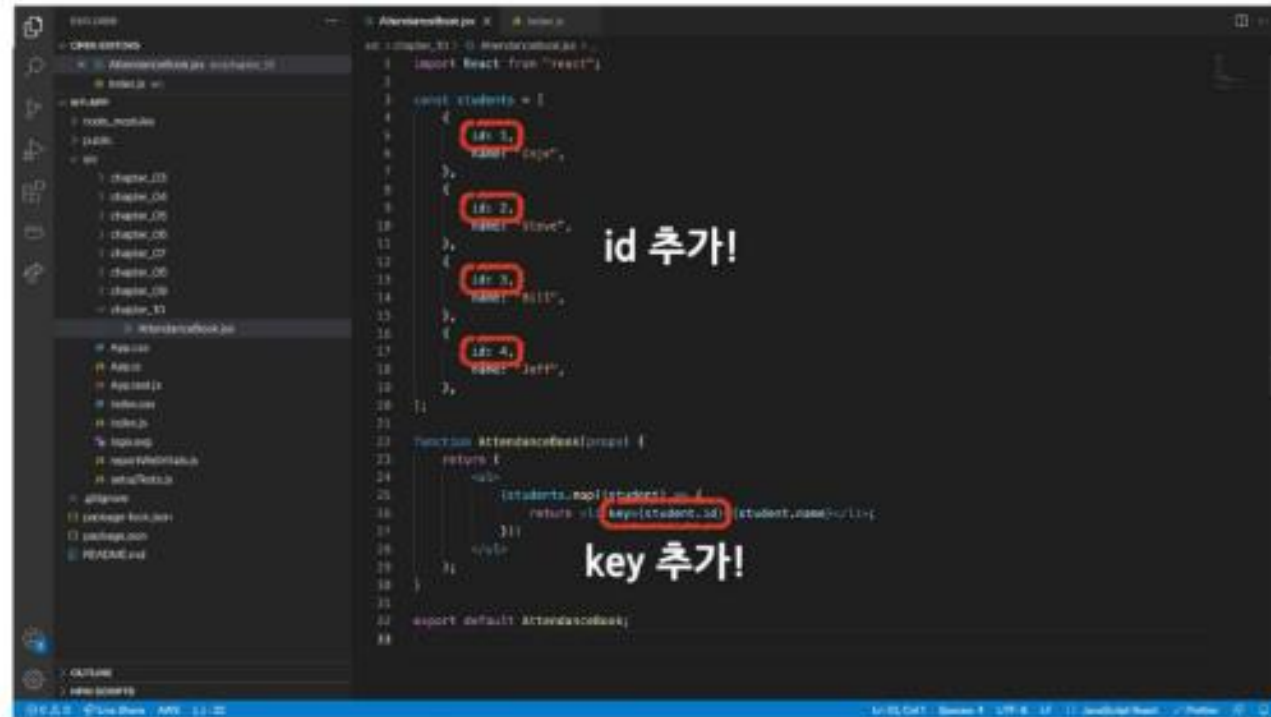
이를 고치기 위해서 아래 그림과 같이 각 학생 객체에 고유한 값을 가진 id를 추가해 주고 `map()` 함수의 엘리먼트에 `key={student.id}`를 넣어줍니다. 이렇게 하면 학생의 아이디가 키값으로 사용됩니다.

```
01 import React from "react";
02
03 const students = [
04   {
05     id: 1,
06     name: "Inje",
07   },
08   {
09     id: 2,
10     name: "Steve",
11   },
12   {
13     id: 3,
14     name: "Bill",
15   },
16   {
17     id: 4,
18     name: "Jeff",
19   },
20 ];
```




```
22  function AttendanceBook(props) {  
23      return (  
24          <ul>  
25              {students.map((student, index) => {  
26                  return <li key={student.id}>{student.name}</li>;  
27              }}}  
28          </ul>  
29      );  
30  }  
31  
32  export default AttendanceBook;
```

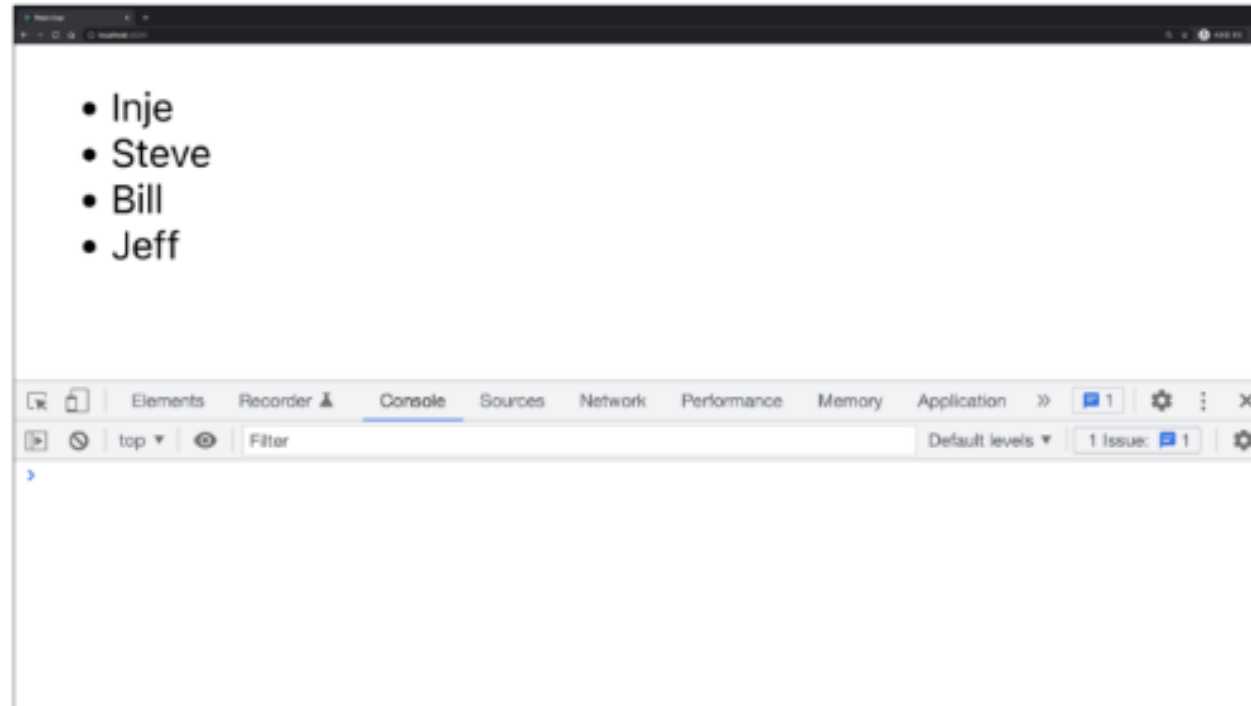




▶ 실습 화면 07

이후에 페이지를 새로 고침하면 다음 그림과 같이 경고 문구가 사라진 것을 볼 수 있습니다.





▶ 실습 화면 08



그리고 위에서 배운 것처럼 키값은 아래와 같이 다양한 방식으로 사용할 수도 있습니다.



```
01 // id를 키값으로 사용
02 {students.map((student) => {
03     <li key={student.id}>{student.name}</li>;
04 }}
05
06 // 포매팅 된 문자열을 키값으로 사용
07 {students.map((student, index) => {
08     <li key={`student-id-${student.id}`}>{student.name}</li>;
09 }}
10
11 // 배열의 인덱스를 키값으로 사용
12 {students.map((student, index) => {
13     <li key={index}>{student.name}</li>;
14 }}
```



요약

• 리스트

- 같은 아이템을 순서대로 모아 놓은 것

• 키

- 각 객체나 아이템을 구분할 수 있는 고유한 값

• 여러 개의 컴포넌트 렌더링

- 자바스크립트 배열의 `map()` 함수를 사용
 - 배열에 들어 있는 각 변수에 어떤 처리를 한 뒤 결과(엘리먼트)를 배열로 만들어서 리턴함
 - `map()` 함수 안에 있는 엘리먼트는 꼭 키가 필요함

• 리스트의 키

- 리스트에서 아이템을 구분하기 위한 고유한 문자열
- 리스트에서 어떤 아이템이 변경, 추가 또는 제거되었는지 구분하기 위해 사용
- 리액트에서는 키의 값은 같은 리스트에 있는 엘리먼트 사이에서만 고유한 값이면 됨

• 다양한 키값의 사용법

- 숫자 값을 사용
 - 배열에 중복된 숫자가 들어 있다면 키값도 중복되기 때문에 고유해야 한다는 키값의 조건이 충족되지 않음
- id를 사용
 - id의 의미 자체가 고유한 값이므로 키값으로 사용하기 적합
 - id가 있는 경우에는 보통 id 값을 키값으로 사용



- 인덱스를 사용

- 배열에서 아이템의 순서가 바뀔 수 있는 경우에는 키값으로 인덱스를 사용하는 것을 권장하지 않음
- 리액트에서는 키를 명시적으로 넣어 주지 않으면 기본적으로 이 인덱스 값을 키값으로 사용

